GentleVanguard

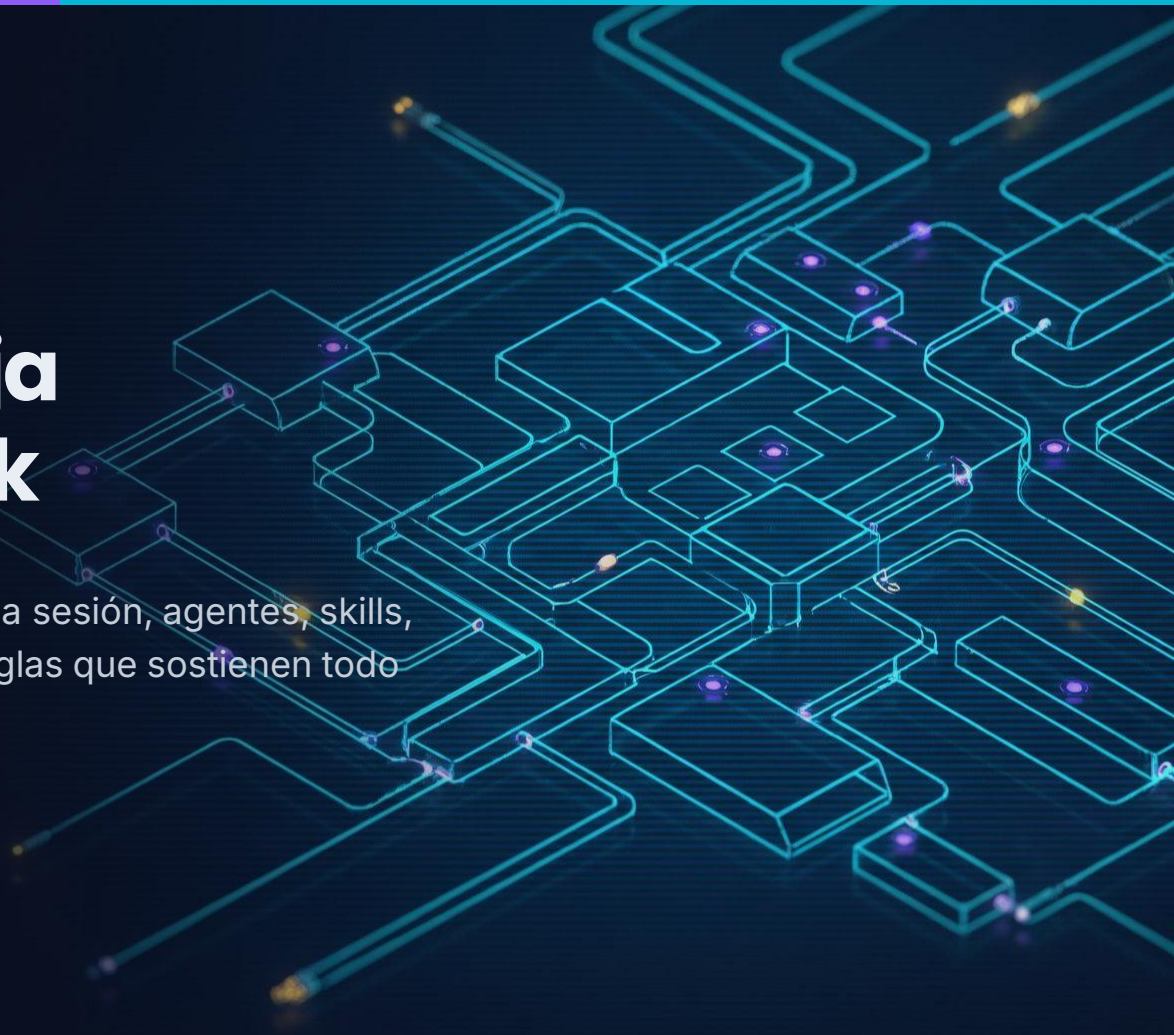
ONBOARDING TÉCNICO

Cómo se trabaja dentro del stack

Mapa del repositorio, anatomía de una sesión, agentes, skills, comandos, base operacional y las reglas que sostienen todo lo demás.

Stack v4.3.0 | Node 24 | pnpm 11

Guía de incorporación - Noviembre 2026



RECORRIDO

Lo que vas a recorrer

Orden recomendado para una primera semana.

01 **Mapa del repositorio**
Qué vive en cada carpeta y por qué está separada.

02 **Anatomía de una sesión**
Qué pasa entre que llegan tu prompt y el agente responde.

03 **Agentes y skills**
Los 21 roles del roster y cómo se invocan las capacidades.

04 **Comandos**
Los comandos que se usan todos los días y qué verifican.

05 **Nexus**
La base operacional: 43 tablas, migraciones y respaldo.

06 **Flujo de desarrollo**
Del pedido al commit: ODD, slices, gates y frontera revisada.

07 **Reglas del repo**
Lo que no se negocia y por qué existe.

08 **Dónde buscar**
La ruta más corta a cada tipo de respuesta.

MAPA

Ocho carpetas, una responsabilidad cada una

La separación no es estética: es lo que permite que las validaciones sean automáticas.

src/	786 archivos TypeScript. Código del stack por área: orquestación, seguridad, retrieval, datos, tools, quality y ops.
apps/	28 aplicaciones propias que operan sobre el stack y validan la arquitectura en producción.
skills/	250 capacidades con contrato y frontmatter. Se invocan por descripción de tarea.
docs/	Manual de stack por módulos, manual completo, 38 ADRs y registro de decisiones de diseño.
config/	Fuente única de configuración: router de modelos, mapeo de roles, quality gates, budgets y registro MCP.
packages/	Workspace con el design system oficial y paquetes compartidos que el código importa.
tests/	272 archivos de test: unit, e2e, integración, seguridad, skills, workflows, eval y performance.
assets/	Marca oficial, tipografías self-hosted y diagrama del shell canónico.

SESIÓN

Qué pasa cuando escribís un pedido

Seis etapas, todas instrumentadas. Ninguna es magia: todas dejan registro.

Autostart

Prepara sesión, engram, orquestador de seguridad, codegraph, budgets, watchtower y Nexus.

>

Preproceso

Inyecta skills, contexto y reglas antes de que el modelo vea el mensaje.

>

Router

Elige modelo por costo, latencia, dominio y criticalidad de la tarea.

>

Agente

Ejecuta con herramientas reales. Cada paso queda en la traza.

>

Skills

Invocadas por contexto. Cada skill declara su contrato y su resultado.

>

Nexus

Sesiones, trazas, tokens, eventos y outcomes de skills quedan persistidos.

21 agentes, cada uno con su perímetro

Un agente tiene modelo, permisos, límite de pasos, presupuesto y fallback declarados.

Orquestación

orchestrator, session-agent, knowledge-agent y self-diag-agent: el loop y su propio diagnóstico.

Ciclo SDD

sdd-explore, sdd-design, sdd-apply y sdd-verify: la vía opt-in para specs de varias sesiónes.

Operación

ops-agent, maintenance-agent, gitflow-agent, premortem-agent y doc-agent: la operación diaria.

Áreas

mkt, sales, finance, hr, legal, bus-tele y sia: roles de negocio con permisos propios.

CAPACIDADES

Skills: la biblioteca que evita reescribir

Qué son

Un SKILL.md con frontmatter, descripción y contrato. Se indexan y se enrutan por similitud de tarea, no por que alguien se acuerde del nombre.

Qué no se copia

Solo 28 skills críticas se propagan a las cinco superficies externas. El resto no se copia para no degradar la metadata del host.

Cómo se invocan

El router de skills puntúa la tarea contra la descripción de cada capacidad y propone la más cercana.

Adoptadas

Incluye frontend-design, canvas-design, doc-coauthoring, theme-factory y la familia de marketing, con atribución en el frontmatter.

Cómo se validan

24 contratos críticos se prueban en ejecución real. Nexus separa outcomes de producción, evaluación y mantenimiento.

Higiene

Cada skill pasa por lint de estructura, de acentuación y por el escaner de secretos antes de entrar al índice.

Los que se usan todos los días

Cada operación del stack es un comando auditable. No hay acciones ocultas.

Comando	Que hace
npm run session:autostart:detached	Arranque de sesión en background. Idempotente. ~1,3 segundos.
npm run watchtower:health	154 checks sobre 29 componentes. PASS esperado.
npm run db:health	Verificación de Nexus: integridad, WAL, tablas y filas.
npm run token:status	Presupuesto real: usado, techo y porcentaje.
npm run stack:facts	Regenera el inventario de hechos del stack y su documentación.
npm run stack:facts:check	Falla si hay drift estructural respecto del inventario.
npm run quality:universal	Quality gate universal: intake, plan, build, verify, release, operate, learn.
npm run guard:snapshot	Respalda untracked y parches sin commit antes de una operación riesgosa.
npm run scan:secrets -- --scan	80 patrones de detección, con entropía opcional y redacción.
npm run graphify -- query "..."	Consulta al grafo AST determinista del repositorio.
npx tsx src/orchestration/recommend-agent.ts --task "..." --topn 3	Recomendación de agente para una tarea.
npm run upgrade	Sincroniza el workspace con lo publicado en origin, con snapshot previo.

Nexus: la memoria operacional del stack

43 tablas y 22 migraciones en SQLite con WAL. Todo lo que el stack aprende queda acá.

Qué guarda

Métricas, sesiones, trazas, eventos, alertas, feedback, cache, uso de skills, tokens y routing.

Retención

Eventos a 30 días, cache a 7 y uso de tokens a 90. Backups diarios con 14 copias retenidas.

Consultas

SQL directo sobre la base, sin capa de reporte que pueda desincronizarse de la verdad.

Integridad

El watchtower vigila tamaño de archivo, WAL, integridad y recuento de filas en cada ciclo de salud.

Engram	Memoria persistente entre sesiones
CodeGraph	Símbolos y relaciones del repositorio
Nexus	Métricas, eventos y trazas (SQLite)

CALIDAD

La calidad no se pide: se ejecuta

El stack corre sus propios controles en cada commit, en cada push y en cada ciclo de salud.

30

Quality gates

28 de 30 aplicados de forma estricta

272

Archivos de test

Unit, e2e, integración, seguridad y skills

21

Workflows de CI

CI, seguridad, release, sync público y etiqueta automática

80

Patrones de secreto

Con entropía opcional y redacción por defecto

Del clone a la primera sesión

Pasos idempotentes: si se corta, se vuelve a correr sin duplicar nada.

01

Prerrequisitos

Node 22.18 o superior, pnpm 11 y Git. Verificación estricta con install:doctor.

02

Dependencias

pnpm install con lockfile congelado. El árbol del workspace se resuelve una vez.

03

Nexus

Se crea la base SQLite en .runtime y se aplican las migraciones pendientes.

04

Hooks

lefthook queda instalado con los hooks de commit y push.

El router decide, y deja decidir el código

La temperatura y el modelo por defecto están bloqueados por política: no se cambian en caliente.

Criterio	Regla vigente
Perfiles	cheap, balanced y premium. El activo es balanced.
Por dominio	premium para finanzas, legal y gobernanza; balanced para marketing, ventas, RRHH y telecom; fastCheap para gitflow, ops y session.
Escalamiento	Sube de nivel por longitud de entrada, cantidad de ítems o preguntas de diseño arquitectónico o seguridad.
Failover	Cadena opencode, ollama, lm-studio2, con cache de 5 minutos y modelo de respaldo.
Circuito	Cinco fallas abren el circuito, dos exitosos lo dejan en half-open.
Temperatura	Bloqueada por defecto. La modificación por comando no está permitida.
Auditoría	Cada decisión de ruteo queda en un log de 90 días de retención.

REGLAS

Lo que no se negocia en este repositorio

Procesos invisibles

Todo spawn pasa por runNpxTsx con windowsHide. Cero ventanas de consola. Hay un test de regresión que lo verifica.

Documentación con orden

Resumen, contexto, alcance, fuera de alcance, supuestos, solución, operación, datos, seguridad, plan y anexos. Nunca "TBD".

Guardrail de destrucción

No se borra ni sobrescribe masivamente sin confirmación explícita. El snapshot de untracked corre antes de cada operación riesgosa.

Commits por slice

Trabajo grande: feature-doc antes del primer write, slices de unas 400 lineas y un commit convencional por slice.

Solo TypeScript y bash

La lógica del stack va en TS. PS1 y CMD solo en excepciones sancionadas. No se crean scripts nuevos en PowerShell.

Frontera revisada

Tras una revisión aceptada, la siguiente arranca desde el delta y no desde el rango completo.

Dónde buscar cada tipo de respuesta

El conocimiento está versionado. No hace falta preguntar.

Necesito	Ir a
La regla vigente del stack	AGENTS.md en la raiz del repositorio
El detalle de un modulo	docs/stack-manual-modules.md
La arquitectura completa	docs/stack-manual-full.md
Cifras generadas del stack	docs/generated/STACK-FACTS.md
Ejemplos de tareas con agentes y skills	docs/use-cases/EXAMPLES.md
Plantillas de proyecto, propuesta y contrato	docs/templates/gv/
Cómo está el design system	packages/gv-design-system y docs/brand/
El libro de presentaciones interactivo	docs/presentations/

GentleVanguard

Tu primer día

Instalá, arrancá una sesión y pedí el estado de salud del stack. Después elegí un issue chico y dejá que el flujo inline haga el resto: explorar, implementar y verificar en proporción al riesgo.

Empezar por setup

Ver el mapa de comandos

Documentación: AGENTS.md y docs/stack-manual-modules.md

Comando de arranque: npm run setup

Estado de salud: npm run watchtower:health

Ante cualquier duda sobre una regla del repositorio, la respuesta esta en AGENTS.md o en el manual de stack.